



第15章： MoE (Mixture of Experts) 与可扩展训练/推理系统



学习目标与主线

学习目标与主线

01

理解 MoE 里 “算法—系统” 耦合：路由如何决定通信、带宽与尾延迟

02

主线框架：Router → 并行通信 (EP / all-to-all) → 稳定训练 (LB / z-loss / capacity) → 低延迟推理 (decode 优先)

03

落地能力：能在质量、吞吐、显存与 p95/p99 之间做设计权衡与系统选型



MoE 基础与设计动机

MoE 与可扩展训练/推理系统

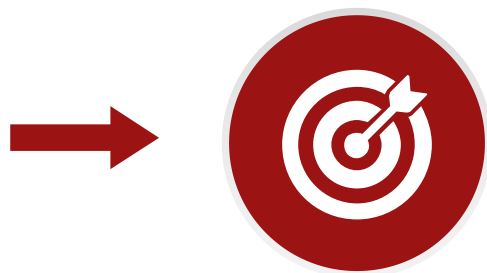
稀疏激活

➤ 核心:

MoE 让总参数大幅增加, 但每个 token **只激活** Top-k 个专家, 所以每 token 计算量 (activated FLOPs) **可控**

➤ 直觉对比:

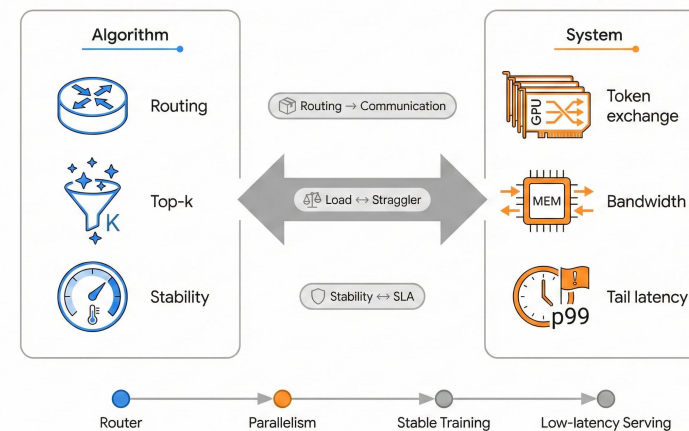
- Dense 扩参: 参数↑ ⇒ 每 token 计算/显存/带宽几乎都跟着↑
- MoE 扩参: 参数↑ (更多专家) 但 每 token 只走少数专家 ⇒ “算力账” 没线性爆炸



难点不在数学, 而在系统

MoE 的 “额外账单”

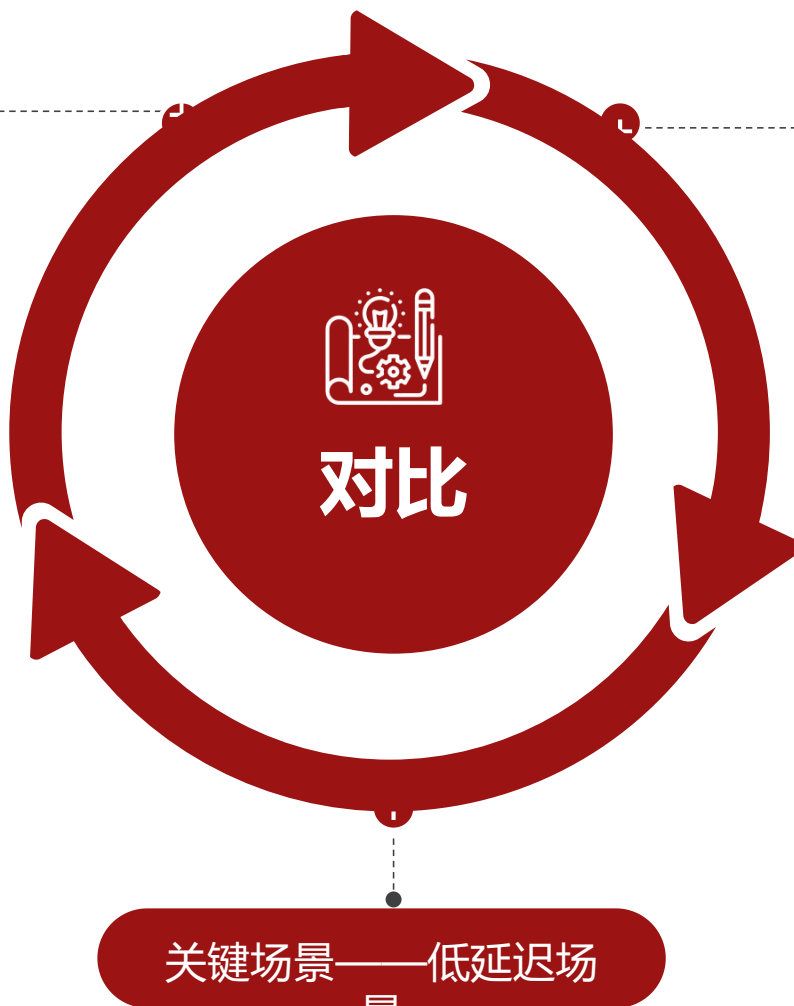
- 通信: 专家分布在不同 GPU 上, token 得 “发快递” 到对应 GPU 再算 (全互换)
- 带宽: 专家权重大、访问更随机, 常被 “读显存速度” 卡住 (算力还没跑满)
- 尾延迟: 最拥堵的那张卡 / 最慢那条链路拖后腿, 决定 p95/p99 (木桶效应)



设计动机：FLOPs↓之后，瓶颈转向带宽/通信

Dense 扩参

- Dense 变大 $\approx$ 每层权重都变大 $\rightarrow$
 - 计算量 (FLOPs) $\uparrow$ (矩阵更大)
 - 显存占用 $\uparrow$ (参数/激活更大)
 - 读权重的带宽需求 $\uparrow$ (每个 token 都要读全量层权重)
- 直觉：
“全量参数参与的逐层计算”——对每个 token，在每一层都需要访问并计算同一套完整权重矩阵（尤其是 FFN/MLP 的大矩阵乘）。
因此参数一放大，权重读取带宽需求、矩阵乘 FLOPs、以及参数/激活显存占用都会近似按规模同步上升，推理/训练时间也随之被这些资源约束“线性拉长”



MoE

Activated FLOPs 可控，但系统账单变重

- MoE 的好处：每个 token 只算 Top-k 个专家 $\rightarrow$ 计算不随总参数线性爆炸 (activated FLOPs 可控)
- MoE 的代价
 - 跨卡交换 (all-to-all)：token 要被“分发”到不同 GPU 上的专家，再把结果“收回”
 - 权重搬运：专家更多、更分散，权重访问更碎片化，容易“读权重慢”
 - 尾延迟放大：最忙的专家或最慢链路会拖住整体 (p95/p99 更敏感)

关键场景——低延迟场

- memory-bound：不是算力不够，而是 读显存/读权重 跟不上 (GPU 算术单元在等数据)
- bandwidth wall：你再加 FLOPs (更强 GPU) 也不明显变快，因为瓶颈卡在 HBM/NVLink/网络带宽上

MoE 的扩展不是“更大算力”就能解决，关键是带宽与通信路径

MoE-FFN: MoE 放在 Transformer 哪里



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

存在位置

- 标准做法: MoE 几乎总是替换 FFN/MLP 子层, 而 Attention 通常保持 dense
- 原因直觉
 - FFN 是每层里参数量最大、计算最规整的部分, 最适合“复制成多专家做稀疏激活”



MoE 核心

Expert

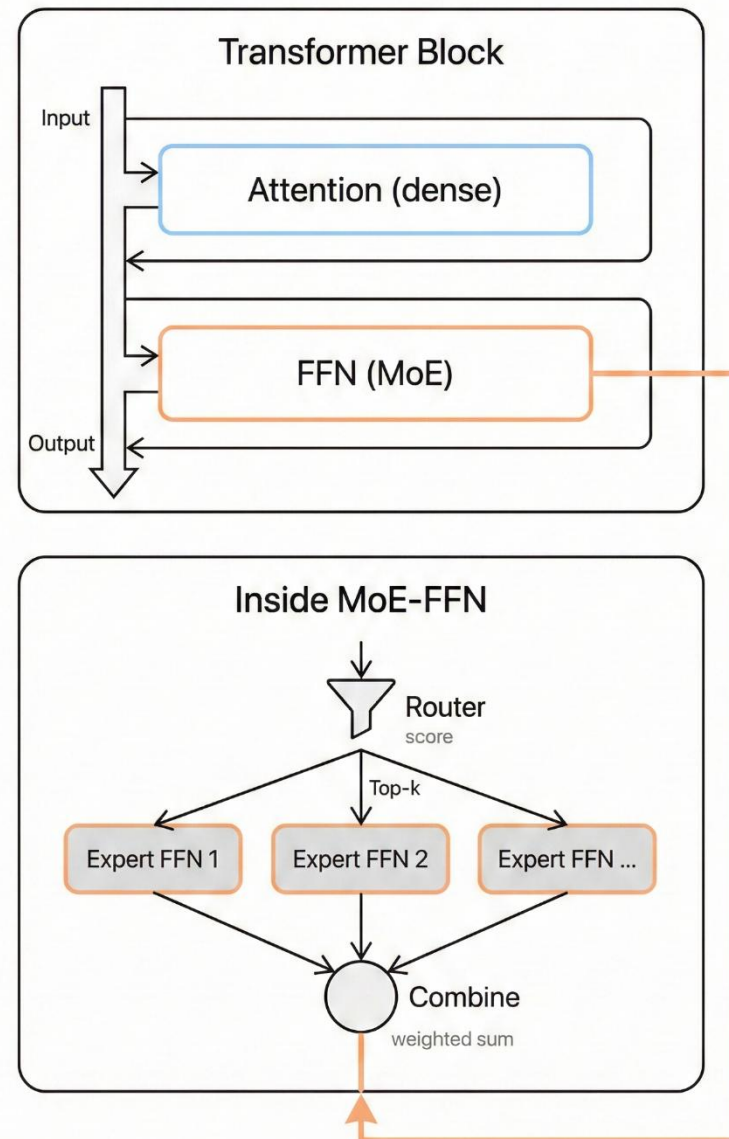
- Expert 到底是什么?
 - Expert = 一份 FFN (MLP) 模块的复制体 (同结构、不同参数)
 - 所谓“多专家并行”, 本质是: 模型里有很多份 FFN 候选, 但每个 token 只走其中 Top-k 份

合并输出

- 输出如何合并回主干
 - 每个被选中的专家输出一个 FFN 结果 y_i
 - 用 Router 权重做加权合并

$$y = \sum_{i \in \text{Top-}} w_i \cdot y_i$$

- 这个 y_i 会像普通 FFN 输出一样, 继续走残差连接 + LayerNorm, 回到 Transformer 的主 token 流



现代 MoE: Shared + Routed + Fine-grained



国产AI亮点

以 DeepSeek 为例

- **做法:** 在 MoE-FFN 里留一条共享通道，一小组 Shared experts 对所有 token 都执行（不经过 Top-k 选择）

- **作用:** 承载通用知识/通用变换+训练更稳

Shared experts

通用能力的“稳定底座”

- **做法:** Router 对每个 token 打分，从一大池 routed experts 里选 Top-k 个执行

- **作用:** 让不同 token 走不同专家，形成功能分工/领域专门化+保持激活计算量可控

Routed experts

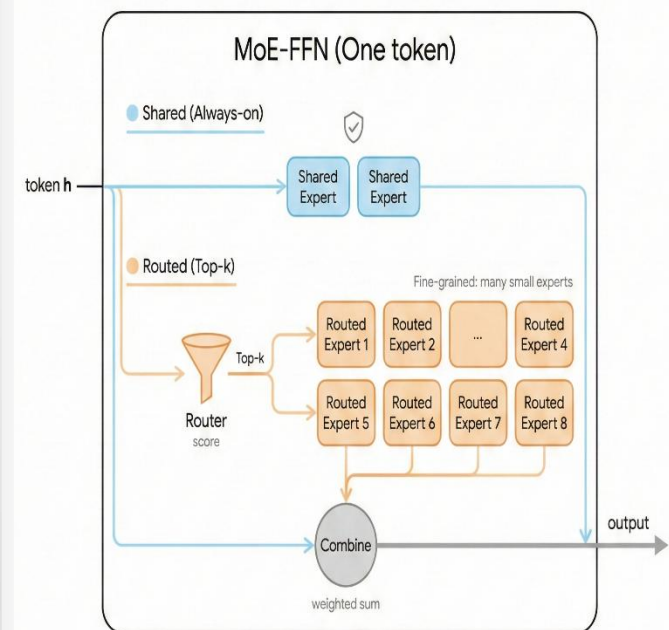
专门化能力的“可选择模块库”

- **做法:** 不是少量“大专家”，而是把 FFN 拆成更多、更小的专家（expert 数↑，单 expert 宽度/容量↓）

- **作用:** 计算几乎不变的情况下可选择/可组合的模块数暴增 ⇒ 更容易形成细粒度分工

Fine-grained

同样算力下做大“可组合空间”



现代 MoE = “底座通用 + 路由专门化 + 细粒度组合”



Router 与路由策略

Router/Gating: 从打分到 Top-k

1) 每个 token 都要先“打分”

- 输入: token 的隐藏表示 h (来自上一层/上一子层)
- Router 是一个很轻量的映射 (通常线性层/小 MLP), 输出对每个专家的 logits
$$s = W_r h$$
- 把 logits 变成概率 **probs** (常见 softmax)
$$p = \text{softmax}(s)$$

打分

计算

2) Top1 / Top2 / Top-k: 只让少数专家参与计算

- 从 p 里选出最大 k 个专家索引: Top-k(p)
- Top1: 最省算、最省通信, 但更容易负载不均/塌缩
- **Top2**: 更稳、更常见, 但计算与通信更重
- **Top-k**: k 越大越稳, 但系统成本越高 (尤其 decode 时)

归一化

3) Renorm: 只在被选中的专家里“重新归一化”

- 只保留 Top-k 的概率, 其余置零, 然后对剩余权重做归一化
- 目的: 让最终合并是一个稳定的加权和 (权重和为 1), 避免数值漂移

5) 训练 vs 推理: Router 策略可以不同 (不同的温度/噪声/采样)

- **训练阶段** (更关注探索与负载均衡):
 - 温度 T : 调软/调硬分布, 影响专家多样性
 - 噪声 (如在 logits 上加噪): 鼓励探索, 避免早期塌缩
 - 采样/随机性: 让不同 token 更均匀用到专家
- **推理阶段** (更关注确定性与延迟):
 - 通常更偏向 **确定性的 Top-k** (少噪声/固定策略)
 - 目标是稳定延迟与可复现输出

训练推理

Router
Gating

加权合

4) 把多个专家的结果写回 token 流

- 每个被选中专家输出 y_i , 然后最终输出:

$$y = \sum_{i \in \text{Top-k}} w_i \cdot y_i$$

- 这一步决定了 MoE 的“语义”: token 不是选一个专家, 而是选一组并加权融合 ($k=1$ 时退化为单专家)

Top1 vs Top2: 质量—稳定性—系统成本

Top1: 最省算、最省通信, 但更容易“偏科”

1

系统收益: 每个 token 只发往 1 个专家

- 计算量最低 (只跑 1 次 FFN)
- 通信量最低 (dispatch/return 都少)
- 专家侧 batch 更集中, 实现上更简单, 吞吐通常更高

2

训练风险: 路由更“硬”, 更容易出现

- 负载不均: 热点专家被挤爆、冷门专家几乎没梯度
- 路由塌缩: 少数专家长期垄断, 模型有效容量下降

3

质量层面: 在分工还没形成/数据多样性大时, Top1 更容易不稳 (对稀有模式覆盖差)

Top2: 更稳、更常用, 但成本明显更高

1

更稳: 每个 token 同时走 2 个专家, 相当于“软集成/冗余路径”

- 对早期训练的探索更友好, 降低塌缩概率
- 对负载波动更有缓冲 (一个专家拥堵/学得不好, 还有另一个兜底)

2

系统代价

- 计算 $\approx \times 2$ (两次 expert FFN)
- 通信更重 (token 要发给两个专家并回收两份输出)
- 尾延迟更敏感: 两条路径里“更慢的那个”可能决定 token 完成时间 (对 p95/p99 更不友好)

3

选型经验: 训练偏 Top2, 线上回 Top1

- 训练阶段: 更关注稳定收敛与专家分工形成 → Top2 更常用 (配合 LB loss / z-loss)。
- 在线推理: 能接受质量下降就用 Top1 换更低延迟/更低通信

总结

Top1 vs Top2 是“质量/稳定”对“系统成本”的经典 **trade-off**



并行通信与训练稳定

Expert Parallel (EP) : MoE 的系统本质

专家分布在不同 GPU/节点:

- **核心概念**: MoE 的专家参数不在一个单一 GPU 上, 而是**跨多个设备** (如多 GPU 或多节点) 进行分布, 通常是分片/切分 (每个 GPU 或节点负责一部分专家)
- **本质**: MoE 是通过将计算与存储分布在不同设备上来实现可扩展性, 不是一个简单的模型并行方案, 而是专门化的“专家分布式计算”

不同的提示结构

Straggler: 最慢的专家/链路决定 p99

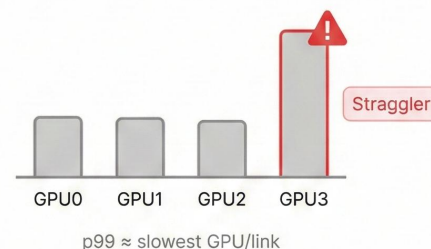
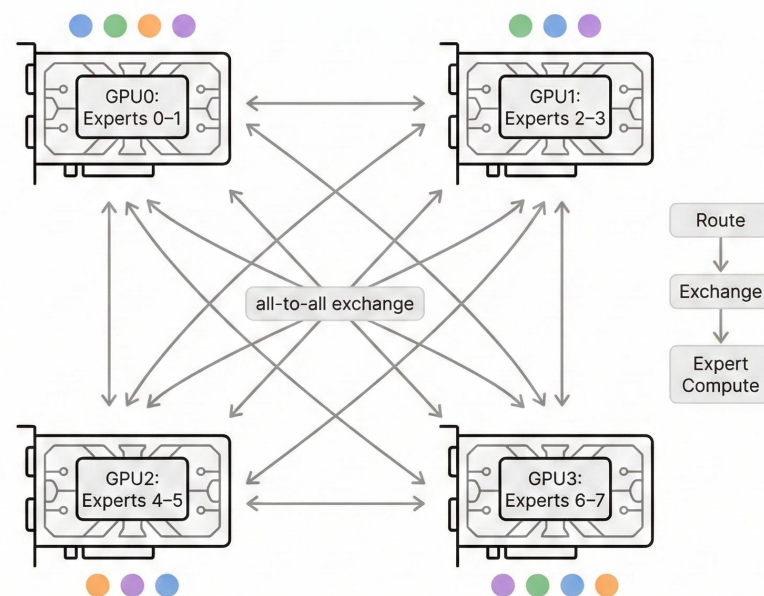
- **Straggler (拖后腿者)**: 在 MoE 系统中, straggler 是指那些处理速度慢的专家或通信链路
- **延迟瓶颈**: 因此, p99 延迟很可能会由最慢的专家或链路决定, 这个问题在分布式计算中非常常见

Token 按专家重排并跨设备交换: all-to-all 通信

- **工作原理**: 当一个 token 被输入时, 它需要根据 Router 决定该选择哪些专家
 - 路由策略 (如 Top-k) 决定了该 token 需要去哪些 GPU/设备的专家上执行计算。这就需要在各个设备之间进行跨设备的数据交换
 - all-to-all 通信: token 需要与所有设备进行交互, 分发到相应的专家进行计算, 计算结果再汇聚回主线程。这是 MoE 的**计算开销和带宽瓶颈**之一

性能优化挑战: 减少 Straggler 影响

- **专家负载均衡**: 确保没有单个专家成为 bottleneck
- **通信优化**: 减少跨设备的通信延迟, 使用更高效的通信协议或硬件 (如 高带宽内存/网络)
- **系统架构设计**: 合理分配专家到设备, 避免某些设备长时间处于过载状态



Dispatch/Combine: 从通信到 kernel 的关键路径

一句话定位

MoE 的性能不只看“选了哪个专家”，更看这条流水线能不能把 token 送对地方、把专家侧 batch 吃饱、再把结果无损送回去

- **输入：**一批 token hidden states + Router 的 Top-k 选择结果 (token→expert 映射)
- **关键动作：**
 - 按专家分桶 (bucketing)：把属于同一专家的 token 聚在一起，形成每个专家的 mini-batch
 - 重排/压缩索引：记录每个 token 原来的位置（用于后面回写）
 - 跨卡发送 (EP all-to-all)：如果该专家在别的 GPU，上一步的 token 桶会被“打包发过去”

- **本质：**每个专家就是一个 FFN，所以核心算子是 GEMM（矩阵乘）
- **关键差异：**不是一个大 batch，而是 多个专家各自的小 batch (per-expert mini-batch)
- **挑战：**
 - batch 太小 → GEMM 吃不饱，效率差 (kernel launch/内存访问开销占比高)
 - 不同专家 token 数不同 → 负载不均，慢专家拖尾延迟

- **输入：**各专家输出 + Router 权重 (Top-k 的权重)
- **关键动作：**
 - 回传：把专家输出从各 GPU 发回 token 的原归属 GPU（也是 all-to-all 的逆过程）
 - 加权合并：对 Top-k 输出做 weighted sum
 - 按索引反向重排 (un-permute)：把结果放回原来的 token 顺序，接回主干计算图

1) Dispatch: token → 对应专家设备

2) Expert GEMM: per-expert mini-batch 计算

3) Combine: 结果回传并重排回原序

先进实现：dropless / 块稀疏 kernel

- dropless 思路：尽量“不丢 token”，而是用更适配的 kernel 把不规则 token 计算做高效
- 代表：MegaBlocks（强调把 MoE 的不规则性变成更规则的块运算）

混合并行全景: Dense 层 TP/SP, MoE 层 EP all-to-all



重点

混合并行系统是如何把**不同的并行策略** (TP/SP/CP vs EP) 结合, 达到大规模 MoE 模型的**高效计算**

如何使用 TP (tensor parallelism) 来切分参数?

- **TP**: 指**张量并行** (Tensor Parallelism), 即每个设备只存储部分参数, 每个设备负责计算部分参数 (例如, 大型模型的每一层都可以被“拆成多块”)
 - 在 **Dense 层** (比如 Transformer 中的 Attention 和 FFN) 或 MLP 层中, 通常会使用 TP 切分参数。这样, 模型的每个层 (尤其是大型线性层或 FFN 层) 会在多个设备间分摊权重, 使得每个设备计算模型的不同部分
- **SP** (Sequence Parallelism) 和 **CP** (Checkpoint Parallelism): 分别针对 **输入序列的切分**和 **中间激活的切分**, 进一步优化每个设备的计算负载

如何同步高层次的副本和状态?

- **DP (Data Parallelism)**: 每个设备有一个**完整的模型副本** (或部分参数), 并通过 all-reduce 或 all-gather 进行数据同步, 确保每个设备上都有相同的权重副本
- **FSDP (Fully Sharded Data Parallel)**: 在更高层次上, 模型的每个参数都被分片并分布到多个设备上, **不再**在单个设备上保存整个模型副本, 这样可以进一步降低内存压力并提高吞吐率
- 在高层同步 (如在**梯度更新**时), 这些方法确保了多个设备间对**权重**的同步, 即全副本/分片同步, 但在参数计算层面上采用更多的**切分策略**

如何将 EP all-to-all 加入到原本的并行架构?

- **MoE 层**: 将 EP (专家并行) 模式与上述并行策略结合, 在 token dispatch 过程中引入 all-to-all 通信关键动作
 - EP all-to-all: 在每个 token 路由到 Top-k 专家的过程中, 通信变成了 token 要与多个设备/专家进行交互, 具有**更高的带宽要求**
 - 在 MoE 层, token 需要在**多台机器之间交换** (all-to-all), 所以每个设备上的计算都必须同步获取从其他设备的专家计算结果

1) Dense/Attention/MLP

2) DP/FSDP

3) MoE 层

难点: 同一模型中同时存在 TP 通信环与 EP all-to-all

- TP 通信: 涉及设备之间对参数的同步 (通常通过 all-reduce 或 all-gather)
- EP all-to-all: token 在多设备间流动, 不仅要计算, 还要与远程设备交换数据
- TP 和 EP 同时存在时, 计算与通信的带宽争夺会引发瓶颈, 特别是在模型大、设备多的情况下, 带宽成为最大瓶颈

训练稳定性：三大坑总览

负载不均

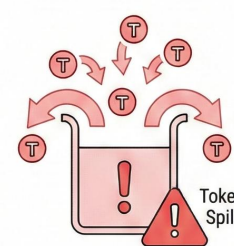
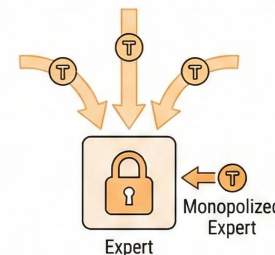
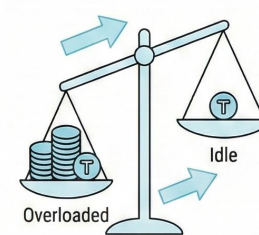
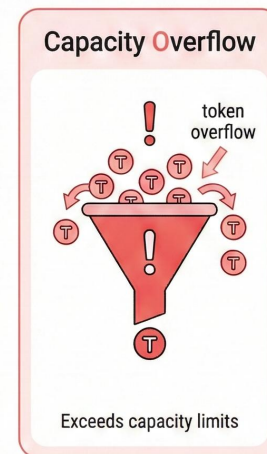
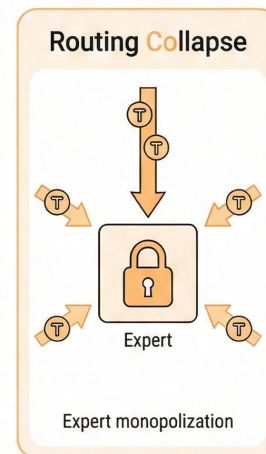
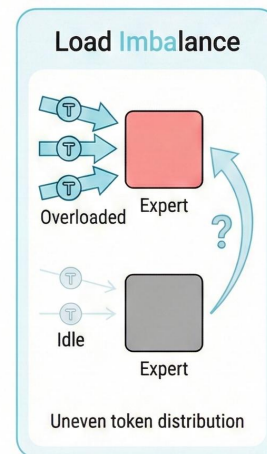
- 热门专家拥塞、冷门专家闲置
- 影响：训练变得不稳定，热门专家过度依赖，可能导致性能下降（如过拟合），而冷门专家未能有效训练，限制了模型的表达能力
- 解决方案：加入辅助负载均衡损失、调节路由策略（如 Top-k）

路由塌缩

- 少数专家垄断
- 影响：模型能力下降+过拟合
- 解决方案：温度调节、噪声注入、路由正则化（例如通过引入 z-loss）来抑制塌缩现象

容量溢出

- 超容量 token 怎么处理
- 影响：可能发生计算溢出，影响整体的训练效率和模型的稳定性
- 解决方案：Dropless 技术（块稀疏（block-sparse）kernel）、专家拆分和负载均衡



负载均衡：Aux Load-Balancing Loss

目标

- 确保每个专家的计算任务 **均匀分配**，避免个别专家过载（拥挤专家）或冷门专家闲置，从而提高训练效率和稳定
- 在 MoE 模型中，每个专家处理不同的 token，但如果专家的负载过于不均：
 - 计算效率低下（一些专家忙得不可开交，其他专家闲置）
 - 梯度更新不充分（冷门专家不能有效学习，最终降低模型的表现能力）

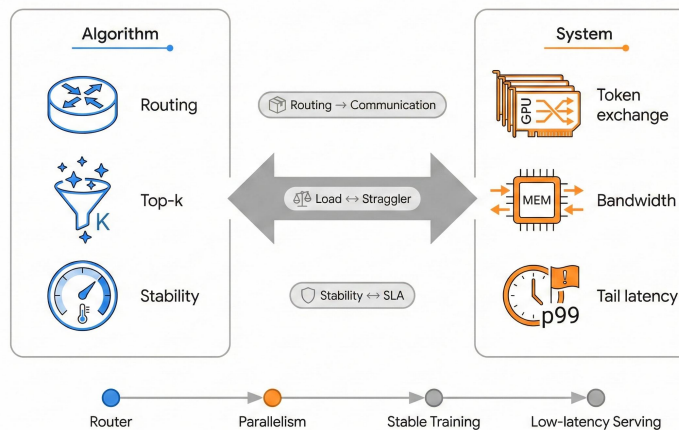


常用信号：

- Usage Fraction：衡量每个专家被调用的频率
- Importance：专家的“重要性”，即每个专家对整体计算结果的贡献度
- 熵（Entropy）：用于衡量专家调用的分布是否均匀。高熵表示专家调用分布均匀，低熵表示存在过度集中或稀疏的情况

Auxiliary Load-Balancing Loss

- Aux LB loss 是 MoE 体系的标配，它通过对路由分布的约束，来促使每个专家的负载尽量均匀
- 如何工作
 - 在计算过程中，会计算每个专家的负载，并为其加入一个**额外的损失项**（Aux LB loss），鼓励路由分布更加均匀



路由塌缩 & 数值稳定: Router z-loss

如何通过 z-loss 来抑制过大 logits, 从而提升训练稳定性

路由塌缩的表现

- 路由塌缩: 是指 token 在 MoE 模型中经过 Router 路由时, 大量 token 被集中地指派给少数专家, 而其他专家几乎没有被激活
- 表现: 这种现象常常表现为 logits (模型给每个专家的分值) 变得极端, 某些专家的 logits 数值非常大, 其他专家则几乎为零, 导致路由分配过于集中

z-loss

- z-loss: 是一种用于数值稳定性和路由多样性的正则化损失函数
- 工作原理: z-loss 会对 logits 进行约束, 让它们的数值不会太极端 (即不允许某些专家的 logits 过大)
- 这种机制在 ST-MoE (一种 MoE 变体) 中得到广泛应用, 能够有效提升训练的鲁棒性和收敛速度

配套技术

- 温度: 通过调节 logits 的“温度”, 使得 logits 不那么极端, 防止路由塌缩的发生, 低温度使得专家之间的选择更加均匀
- 噪声: 给 logits 添加一些噪声, 进一步增加专家选择的多样性
- Warmup: 通过在训练初期逐步增加 logits 的温度或噪声, 防止训练初期路由过度集中

总结

早期通过各种多样化的调控手段、人为干扰, 抑制专家垄断

容量因子与溢出：传统两难到 dropless

容量因子：每个专家接收 token 的上限

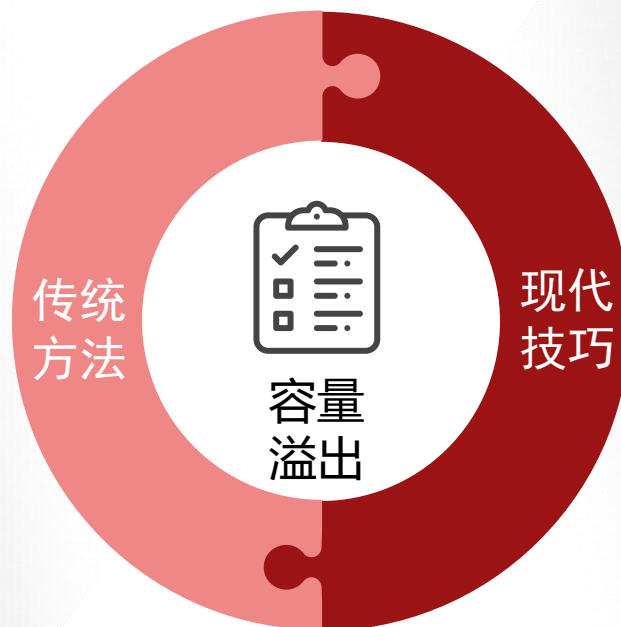
传统的dropping/padding/reroute 方法，虽然能在某些情况下缓解容量溢出问题，但它们各自都有信息丢失、计算浪费和通信负担等问题，影响了模型训练的效率 and 效果

- ✓ **Reroute (重定向)**：通过 reroute (重定向) 策略将超出容量的 token 转发给其他专家处理
- ✓ **Dropping (丢弃)**：超出的 token 会被丢弃
- ✓ **Padding (填充)**：容量不足，会通过一些零值或空白 token来补充空缺

Reroute (重定向)

Dropping (丢弃)

Padding (填充)



Dropless：用块稀疏 (block-sparse) /稀疏 kernel 技术解决“丢 token”问题

- ✓ **块稀疏 (Block-sparse)**：将 token和计算任务分成“块”，只有活跃的块会参与计算
- ✓ **稀疏 kernel (Sparse kernel)**：通过稀疏计算，将计算集中在活跃区域，减少了计算和内存开销，提高了计算效率

块稀疏

稀疏 kernel



低延迟推理与线上工程

推理：Prefill vs Decode（系统代价完全不同）

Prefill (预填充)

概念：

在预填充模式下，系统会在一次推理开始之前，将**整个**输入的**token**批量处理，并进行批量预测

优点：

- 高吞吐量：一次处理大量 token，可以提高系统吞吐量
- 低通信开销：批量计算减少了设备间的通信频率

缺点：

实时性差：由于整个输入需要一次性处理，延迟比较高，不适合实时的应用场景

Decode (逐步解码)

概念：

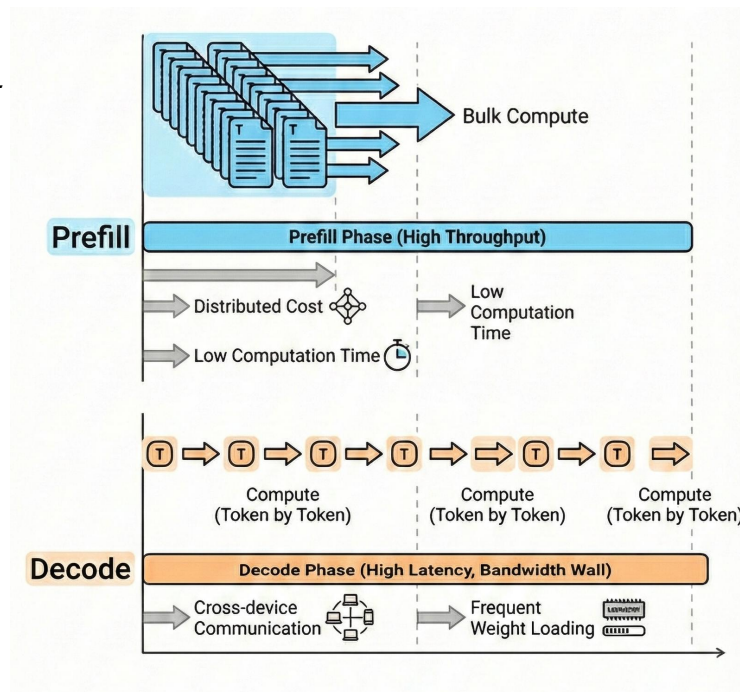
在解码模式下，推理是**逐 token**进行的，系统会根据之前已生成的部分信息逐步解码下一个 token

优点：

- 实时性强：适合实时生成，每次生成一个 token 可以用于实时反馈
- 灵活性高：可以根据每个生成步骤的上下文进行动态调整，适合一些对话生成和其他交互性推理任务

缺点：

带宽压力大：每次生成一个 token 都需要通信和权重读取，带宽成为瓶颈



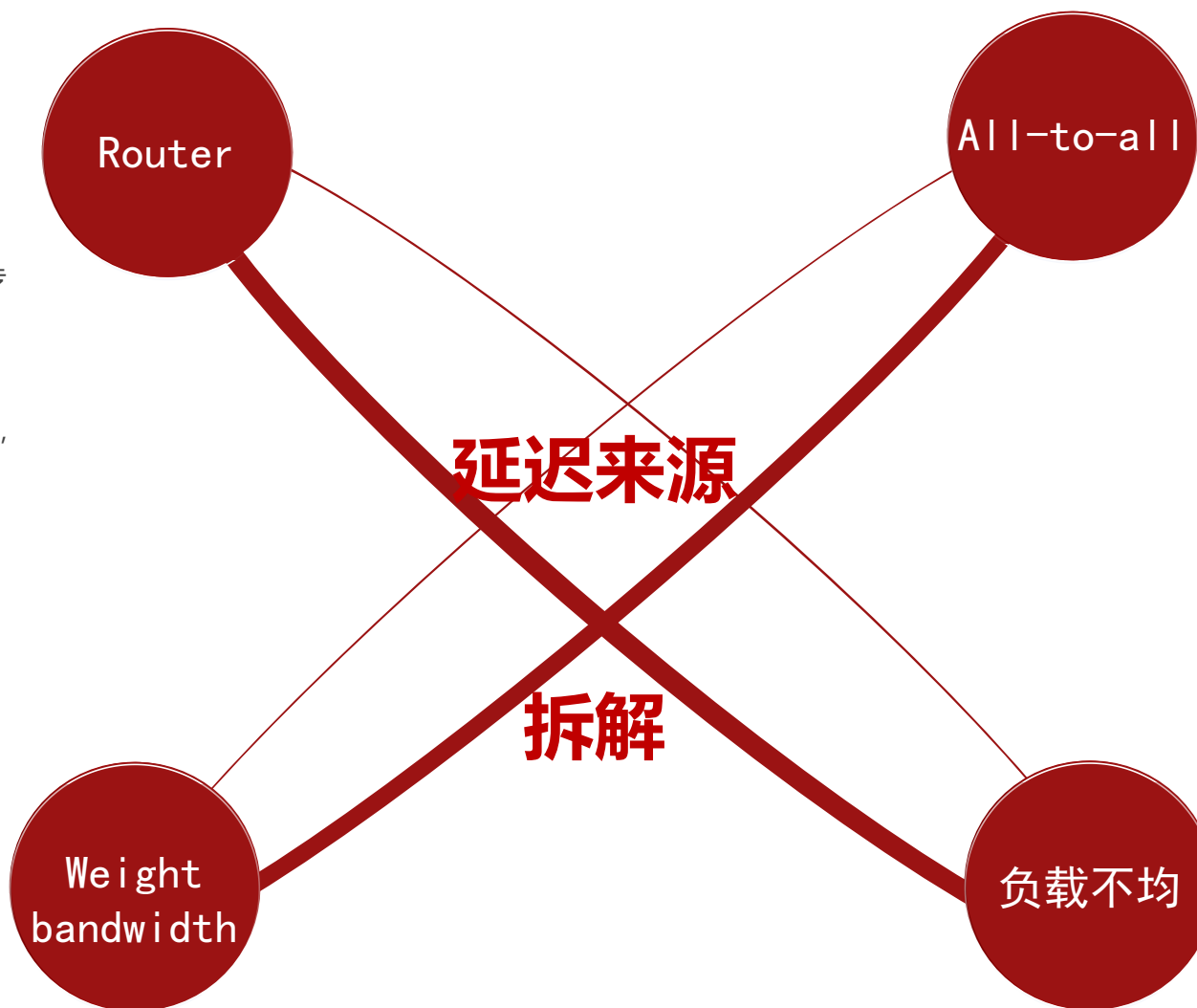
延迟来源拆解：Compute / BW / Network / Straggler



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN



▶ Router (每token必做的额外开销)

- **Router的作用：**是将每个token分配给对应的专家。在这个过程中，Router需要根据每个token的**特征打分**并**选择**Top-k个专家，这个过程需要消耗额外的计算资源
- **额外开销：**体现在：每个token都需要经过Router来决定其应该交给哪些专家处理，这个操作不仅增加了计算量，还带来了通信延迟

▶ 专家权重读取, memory-bound

- MoE中的专家权重存储在GPU的显存中。在推理过程中，每次访问不同的专家时，系统需要读取对应的权重数据，这一过程是memory-bound，即受限于**内存带宽**
- **影响：**如果权重数据过大，或者多个专家需要频繁切换，那么内存带宽可能成为瓶颈，导致读取速度慢，从而影响推理速度

▶ 跨卡/跨机交换与同步

- **all-to-all通信：**是指所有节点（GPU/机器）之间的数据交换。在MoE系统中，这种通信方式常常需要同步大量的模型参数和中间计算结果，尤其是在token的专家分配跨越多个设备时
- **影响：**跨设备的同步和数据交换需要较大的带宽，若带宽不足，可能导致显著的延迟

▶ 最忙专家决定尾延迟

- 在MoE系统中，每个专家可能处理的token数量不同。如果某个专家负载过高，而其他专家空闲，就会出现负载不均的情况
- **影响：**负载最重的专家可能成为瓶颈，导致该专家的处理速度变慢，从而影响整个系统的推理速度。最慢的专家和最慢的网络链路决定了p99延迟，即系统的最差响应时间

Batching 与调度

解释

在MoE系统中，**batching**（批处理）与**调度**（scheduling）是优化系统性能的关键因素，尤其是在多专家（MoE）场景中，合理的batch管理能够显著**提升系统吞吐量**和**延迟**表现

- **动态 batching**：是指在推理过程中，系统根据实时的请求情况，将多个输入合并成一个较大的batch，减少了每个专家处理的batch数量，提升了计算效率
- **拼请求的过程**：意味着当某些token没有足够的请求时，系统会将这些token与其他请求合并，形成一个更大的batch进行处理。这可以提高每个专家的计算效率，因为GPU处理较大的batch通常会更高效，从而减少每个专家在处理token时的等待时间

- 在MoE系统中，专家的计算负载和所处理的token数量是高度不均的。如果token的分布很**不均匀**，就会造成**碎片化**，即某些专家工作过载，而其他专家处于空闲状态
- **Expert bucketing的思想**：是将token**按专家的处理能力或负载**进行分桶，即将**类似的token**分配给同一个专家池，这样可以减少不同专家之间负载的差异。通过这种方式，能够确保每个专家都有足够的工作量，避免了某些专家被过度分配任务，而其他专家却没有足够的负载

- 负载感知的策略意味着系统**实时监控**每个专家的负载情况，避免长期将大量token分配给同一个专家。通过动态调整负载的分配，确保每个专家的负载均衡，减少拥塞并提高整体系统的吞吐量
- 例如，当一个专家的负载达到设定阈值时，系统可以动态改变token的路由策略，选择将一些token路由到负载较低的专家

1) 动态 Batching：拼请求提升专家侧 batch

2) Expert Bucketing：按专家分桶减少碎片化

3) 负载感知：避免热点专家长期拥塞

核心目的：避免过载和闲置，从而提升系统效率和稳定性

显存与权重放置：在线 serving vs 资源受限（明确场景）

01

在线低延迟：权重尽量驻留GPU

- 在低延迟推理中，为了最大限度减少数据读取延迟，通常要求**所有专家的权重**都驻留在GPU显存中
- 好处：可以直接利用GPU的高带宽和并行计算能力，**减少I/O开销**，从而显著**降低延迟**，保证实时性

02

支持部分CPU offload，但性能下降

- **vLLM**是一个在大规模LLM推理中应用的优化框架，它可以通过将部分专家的权重从GPU迁移到CPU来减轻GPU的内存负担，从而支持更大的模型
- 限制：vLLM的设计**不支持**仅加载被门控的少数专家

研究方向

专家缓存+分层存储

- 专家缓存：是指将专家的权重存储在GPU缓存中，只有频繁使用的专家会驻留在GPU内存中
- 分层存储：通过分层存储机制（例如，GPU缓存 + CPU/SSD），系统可以根据访问频率动态调整专家的存储位置

在线低延迟推理与资源受限场景下的**显存与权重放置策略**

量化 + KV 压缩：MoE 的“VRAM 预算”竞争



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

显存和带宽/通信都要进行优化

VRAM

GPU “总预算表”

- Weights (模型权重)：基本是“固定成本”：模型一加载就占住 (MoE 因为总参数巨大，所以权重占用更可怕)
- KV Cache (注意力缓存)：是“可变成本”：随着 $\text{batch size} \times \text{context length} \times \text{layer数}$ 线性增长

GPU 显存想成一个固定容量的“预算池”，推理时主要被两类东西吃掉

DeepSeek

(典型代表)

- 它的核心是把传统注意力里要缓存的 K/V，压缩成更小的“latent 向量”再存起来，从而大幅减少 KV cache
- DeepSeek-V2 的摘要里直接给出了量化结果：KV cache 显著压缩 (并且他们报告了大幅比例的减少)
- DeepSeek-V3 技术报告也继续采用 MLA，并把它描述为用于降低推理时 KV cache 的关键机制

模型量化

(重要性)

- MoE 推理 (特别是 decode) 很多时候不是算不过来，而是**读不过来**
 - 每步要用到专家权重 (尤其跨设备/跨层访问)，权重越大、精度越高，显存带宽压力越大
- 量化把权重从 FP16/FP8/INT8/INT4 等更低比特表示，直接减少
 - 例如 vLLM 的 FP8 文档就明确说：FP8 量化可带来模型内存需求约 $2 \times$ 降低，并提升吞吐

总结

显存是硬约束 (能不能放下)，带宽/通信是软瓶颈 (跑得快不快)
MoE 两个都容易爆，所以要组合优化

Takeaways: MoE 设计与系统协同的三条结论

3

3.线上推理 ▶▶▶

Decode 小 batch 是最难场景：带宽墙 + all-to-all + straggler → p99 延迟优先级最高

2

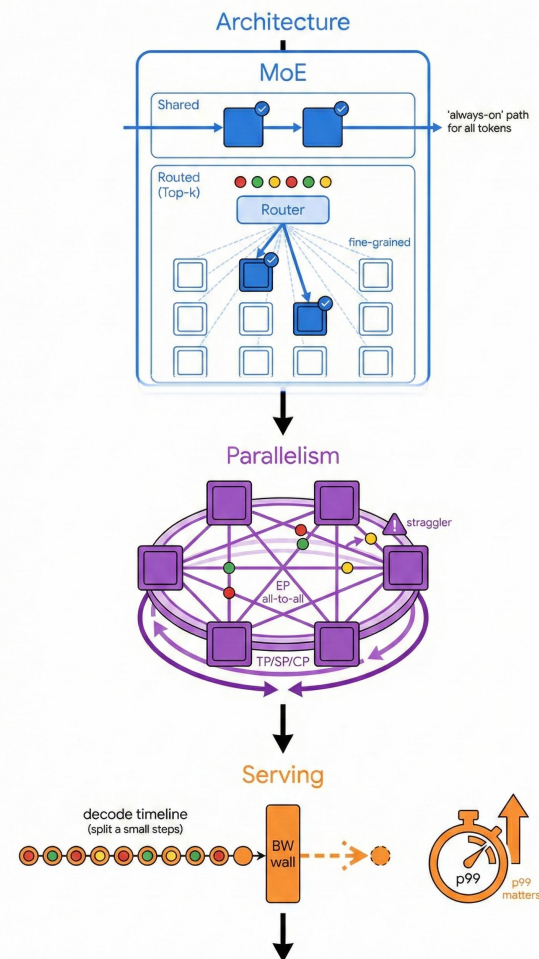
2.系统层面 ▶▶▶

- Dense 层和 MoE 层的通信模式不同，混在一起才是系统难点
- Dense (Attention/MLP) 侧的并行更像“规整的环”
- MoE (EP) 侧的通信是“按路由动态重排”

1

1.架构层面 ▶▶▶

- **Shared experts (共享专家)**：每次都激活，像“通用底座”，保证常识/通用能力稳定输出
- **Routed experts (路由专家)**：按 token 的需求选择 Top-k，像“专科门诊”，负责专业化能力
- **Fine-grained experts (细粒度专家)**：把原本一个大 FFN 切成更多小专家池



01

架构结论：现代 MoE = Shared (通用底座) + Routed (Top-k 专门化) + Fine-grained 专家池，用稀疏激活在不显著增加每 token 计算的前提下扩大“可组合的专门化空间”

02

系统结论：Dense 层的 TP/SP/CP 集体通信与 MoE 层的 EP all-to-all 同时存在，通信与负载不均 (straggler) 共同决定吞吐与可扩展性

03

线上结论：Serving 的第一优先级是 Decode 小 batch 场景：带宽墙 + all-to-all + p99 尾延迟最容易成为瓶颈，因此要组合使用量化/KV 压缩/调度与通信优化